Table des matières

1 Introduction 2

2 Contexte et Justification 2

3 Concepts Clés 2

4 Prérequis 3

5 Points de Vigilance 3

6 Choix Architecturaux & Alternatives 4

6.1 Quand choisir les ACL POSIX 4
6.2 Alternatives et comparatif 4
6.3 Impact des choix 5

7 Configuration 5

7.1 Lire les permissions existantes 5
7.2 Modifier propriétaire et permissions 5
7.3 Régler le umask 6
7.4 Poser les droits spéciaux 6
7.5 Ajouter des ACL nommées 6
7.6 Définir les ACL par défaut 7
7.7 Sauvegarder et restaurer les ACL 7

8 Vérifications 7

9 Dépannage 8

10 Conseils et Bonnes Pratiques 8

11 Procédures Liées 8

1 Introduction

Cette procédure décrit la gestion des droits d'accès aux fichiers sur un serveur Linux Debian 13 ou Ubuntu Server 24.04 LTS : permissions classiques avec `chmod` et `chown`, masque de création `umask`, droits spéciaux SUID, SGID et sticky bit, puis listes de contrôle d'accès POSIX avec `setfacl` et `getfacl`, y compris les ACL par défaut héritées.

Le fil conducteur est un répertoire partagé `/srv/projets` sur srv-lnx-01 (192.168.10.21), utilisé par deux équipes. Les différences avec Ubuntu 26.04 LTS sont signalées quand elles existent.

2 Contexte et Justification

Les permissions Unix classiques ne connaissent que trois classes : le propriétaire, le groupe propriétaire et les autres. Ce modèle suffit pour un répertoire personnel, mais il atteint vite ses limites sur un serveur partagé :

- Deux équipes doivent écrire dans le même répertoire alors qu'un fichier n'a qu'un seul groupe propriétaire.
- Un fichier créé par un membre de l'équipe hérite de son groupe privé, et les collègues ne peuvent plus le modifier.
- Un utilisateur peut supprimer les fichiers des autres dans un répertoire où il a le droit d'écriture.
- Un auditeur doit pouvoir lire un dossier sans faire partie du groupe métier.

Les droits spéciaux et les ACL apportent les réponses à ces cas :

- **SGID sur répertoire :** les nouveaux fichiers héritent du groupe du répertoire.
- **Sticky bit :** seul le propriétaire d'un fichier peut le supprimer.
- **ACL :** des droits nommés pour plusieurs utilisateurs ou groupes sur un même objet, avec héritage par les ACL par défaut.

3 Concepts Clés

Permissions de base. Chaque fichier porte trois triplets `rwx` pour le propriétaire (u), le groupe (g) et les autres (o). Sur un fichier, `r` permet la lecture, `w` la modification, `x` l'exécution. Sur un répertoire, `r` permet de lister, `w` de créer, renommer et supprimer des entrées, `x` de traverser le répertoire.

Notation octale. `r` vaut 4, `w` vaut 2, `x` vaut 1. `750` signifie `rwx` pour le propriétaire, `r-x` pour le groupe, rien pour les autres. Un quatrième chiffre placé en tête porte les droits spéciaux.

umask. Masque retiré des droits maximaux à la création d'un objet (666 pour un fichier, 777 pour un répertoire). Avec un umask `022`, un fichier est créé en `644` et un répertoire en `755`. Sur Debian et Ubuntu, la valeur par défaut est définie par `UMASK` dans `/etc/login.defs` et appliquée par `pam_umask`.

SUID (4000). Sur un exécutable, le processus s'exécute avec l'UID du propriétaire du fichier. C'est le mécanisme de `passwd` ou `sudo`. Le noyau ignore le SUID sur les scripts interprétés.

SGID (2000). Sur un exécutable, le processus prend le GID du groupe du fichier. Sur un répertoire, les fichiers et sous-répertoires créés héritent du groupe du répertoire, et les sous-répertoires héritent du bit SGID.

Sticky bit (1000). Sur un répertoire, seuls le propriétaire du fichier, le propriétaire du répertoire et `root` peuvent supprimer ou renommer un fichier. C'est le réglage de `/tmp` (`1777`).

ACL POSIX. Liste d'entrées supplémentaires de la forme `user:nom:rwx` ou `group:nom:r-x`. Une ACL étendue comporte toujours une entrée `mask` qui plafonne les droits accordés aux utilisateurs nommés, aux groupes nommés et au groupe propriétaire. Un `+` à la fin des permissions dans `ls -l` signale la présence d'une ACL.

ACL par défaut. Posée sur un répertoire avec le préfixe `default:` (ou `d:`), elle n'agit pas sur le répertoire lui-même mais devient l'ACL d'accès de chaque objet créé dedans. C'est le mécanisme d'héritage des ACL POSIX.

4 Prérequis

- Un serveur Debian 13 ou Ubuntu Server 24.04 LTS, ici srv-lnx-01.
- Un compte membre du groupe `sudo`.
- Le paquet `acl`, qui fournit `setfacl` et `getfacl`. Il est installé par défaut sur Ubuntu Server ; sur Debian, vérifiez avec `dpkg -s acl` et installez-le si besoin avec `sudo apt install acl`.
- Un système de fichiers qui gère les ACL POSIX. ext4 et XFS les activent par défaut, sans option de montage.
- Les groupes `grp-projets-dev`, `grp-projets-chef` et le compte `auditeur` créés au préalable.

5 Points de Vigilance

chmod sur un objet avec ACL modifie le masque. Quand une ACL étendue existe, les droits du groupe affichés par `ls -l` et modifiés par `chmod g=` sont ceux du `mask`, pas ceux du groupe propriétaire. Un `chmod 750` peut ainsi réduire silencieusement les droits de tous les utilisateurs et groupes nommés.

Un chmod -R 777 n'est jamais une solution. Il ouvre le contenu à tous les comptes, y compris les comptes de service compromis, et rend les exécutables modifiables. Corrigez la cause (propriétaire, groupe, SGID, ACL) plutôt que d'élargir.

Tous les outils ne conservent pas les ACL. `cp` sans `-a` ou `--preserve=all`, `rsync` sans `-A` et `tar` sans `--acls` les perdent. Un déplacement (`mv`) vers un système de fichiers sans ACL (certains montages réseau) les perd aussi.

Les fichiers SUID root sont des cibles. Chaque binaire SUID `root` est un vecteur potentiel d'élévation de privilèges. N'ajoutez jamais le SUID à un binaire générique (`find`, `vim`, `python3`) et auditez régulièrement la liste.

Les options de montage priment. Un système de fichiers monté avec `nosuid` ignore les bits SUID et SGID sur les exécutables, et `noexec` interdit toute exécution. Debian 13 monte `/tmp` en tmpfs par défaut.

Ubuntu 26.04 utilise rust-coreutils. `chmod`, `chown` et `chgrp` y proviennent de l'implémentation Rust uutils, en cours de stabilisation. Les versions GNU restent disponibles avec le préfixe `gnu` (par exemple `gnuchmod`) ou en installant le paquet `coreutils-from-gnu`. Sur des scripts critiques, validez le comportement avant la mise en production. `setfacl` et `getfacl` ne sont pas concernés.

6 Choix Architecturaux & Alternatives

6.1 Quand choisir les ACL POSIX

Les permissions classiques avec un groupe propriétaire et le SGID suffisent dès qu'un seul groupe doit écrire dans un répertoire. C'est le modèle à privilégier, car il est simple à lire et conservé par tous les outils.

Les ACL POSIX deviennent pertinentes quand plusieurs groupes ont des droits différents sur la même arborescence, quand un compte doit recevoir un accès ponctuel sans rejoindre un groupe, ou quand un héritage automatique des droits est nécessaire. Pour un partage consommé par des postes Windows, la gestion se fait plutôt via Samba, qui peut s'appuyer sur ces ACL.

6.2 Alternatives et comparatif

Option	Cas d'usage typique	Force principale	Faiblesse principale	Coût relatif
Permissions classiques et SGID	Un groupe métier par répertoire	Simple, universel, conservé par tous les outils	Un seul groupe propriétaire par objet	Gratuit
ACL POSIX	Plusieurs groupes aux droits différents, héritage	Granularité, héritage par ACL par défaut	Lisibilité réduite, perte possible lors des copies	Gratuit
ACL NFSv4 / Samba (vfs_acl_xattr)	Partage consommé par des clients Windows	Modèle proche de NTFS (héritage, refus explicite)	Dépend du service de partage, administration à distance	Faible
Contrôle d'accès obligatoire (AppAr	Confinement des services exposés	Limite un processus même s'il tourne en root	Profils à écrire et à maintenir	Moyen

Option	Cas d'usage typique	Force principale	Faiblesse principale	Coût relatif
mor, SELinux)				
Chiffrement par répertoire (fscrypt)	Données sensibles sur poste ou serveur partagé	Protège hors ligne	Gestion des clés, pas un contrôle d'accès en ligne	Moyen

6.3 Impact des choix

- **Sécurité.** Le SGID et le sticky bit corrigent les défauts courants des répertoires partagés sans élargir les droits. Les ACL permettent de respecter le moindre privilège, mais une entrée oubliée reste invisible dans un `ls -l` sommaire.
- **Exploitation.** Les ACL imposent de documenter l'arborescence et d'adapter les outils de sauvegarde et de migration. Les permissions classiques restent lisibles par tout administrateur.
- **Disponibilité.** Une erreur de `chmod -R` ou `setfacl -R` sur une arborescence de production peut bloquer un service. Sauvegardez les ACL avant toute modification récursive.
- **Coût.** Aucun coût de licence. Le coût porte sur la documentation et les revues de droits.

7 Configuration

7.1 Lire les permissions existantes

```
ls -ld /srv/projets
stat -c '%A %a %U:%G %n' /srv/projets
getfacl -p /srv/projets
```

7.2 Modifier propriétaire et permissions

Créez le répertoire partagé et attribuez-le au groupe principal :

```
sudo mkdir -p /srv/projets
sudo chown root:grp-projets-dev /srv/projets
sudo chmod 2770 /srv/projets
```

Le `2` en tête active le SGID : tout fichier créé dans `/srv/projets` appartient à `grp-projets-dev`, quel que soit le groupe principal de son créateur.

Exemples de notation symbolique :

```
chmod u+x script.sh          # ajoute l'exécution au propriétaire
chmod g-w,o= rapport.txt     # retire l'écriture au groupe, tout aux autres
chmod -R g+rX /srv/projets   # X : exécution uniquement sur répertoires et fichiers déjà
exécutables
```

Changez le propriétaire d'une arborescence :

```
sudo chown -R alice.martin:grp-projets-dev /srv/projets/alpha
```

7.3 Régler le umask

Consultez la valeur de la session et testez son effet :

```
umask
umask -S
```

Pour un répertoire collaboratif, un umask `002` permet au groupe d'écrire dans les fichiers créés. Appliquez-le à un service par sa directive systemd plutôt que globalement :

```
sudo systemctl edit mon-service.service
```

```
[Service]
UMask=0002
```

Pour l'ensemble des sessions interactives, modifiez `UMASK` dans `/etc/login.defs`. Avec `USERGROUPS_ENAB yes`, `pam_umask` donne déjà aux bits du groupe les mêmes valeurs que ceux du propriétaire (umask effectif `002`) pour les utilisateurs dont le groupe principal porte leur nom et a le même identifiant que leur UID.

7.4 Poser les droits spéciaux

```
# SGID sur un répertoire collaboratif
sudo chmod g+s /srv/projets

# Sticky bit sur un dépôt commun : chacun ne supprime que ses fichiers
sudo mkdir -p /srv/projets/depot
sudo chmod 3770 /srv/projets/depot      # SGID + sticky + rwxrwx---

# Rechercher les binaires SUID et SGID présents sur le système
sudo find / -xdev -type f \( -perm -4000 -o -perm -2000 \) -exec ls -l {} \; 2>/dev/null
```

> Un `S` ou un `T` majuscule dans `ls -l` indique un bit spécial posé sans le droit d'exécution correspondant. Il est sans effet sur un exécutable et révèle souvent une erreur.

7.5 Ajouter des ACL nommées

Donnez à l'équipe chef de projet un accès en lecture et écriture, et à l'auditeur un accès en lecture seule :

```
sudo setfacl -m g:grp-projets-chef:rwx /srv/projets
sudo setfacl -m u:auditeur:rx /srv/projets
```

Appliquez la même chose au contenu existant :

```
sudo setfacl -R -m g:grp-projets-chef:rwX,u:auditeur:rX /srv/projets
```

Retirez une entrée, ou toutes les entrées étendues :

```
sudo setfacl -x u:auditeur /srv/projets
sudo setfacl -b /srv/projets/archive
```

7.6 Définir les ACL par défaut

Les ACL par défaut garantissent que tout nouveau fichier ou sous-répertoire reçoit les mêmes droits :

```
sudo setfacl -R -d -m g:grp-projets-dev:rwX,g:grp-projets-chef:rwX,u:auditeur:rX,o::---
/srv/projets
```

> Les droits d'un fichier créé résultent de l'intersection entre l'ACL par défaut et le mode demandé par l'application. Un éditeur qui crée en `0644` obtiendra un masque `r--` pour les groupes nommés. Les ACL par défaut remplacent le umask pour les objets créés dans ce répertoire.

7.7 Sauvegarder et restaurer les ACL

Avant toute modification récursive, exportez l'état courant :

```
cd / && sudo getfacl -R -p srv/projets > /root/acl-projets-$(date +%F).txt
```

Restaurez en cas d'erreur :

```
cd / && sudo setfacl --restore=/root/acl-projets-2026-10-16.txt
```

8 Vérifications

Permissions et SGID : `stat -c '%A %a %U:%G' /srv/projets` affiche `drwxrws---+ 2770 root:grp-projets-dev`.

ACL effectives : `getfacl /srv/projets` liste les entrées `group:grp-projets-chef:rwx`, `user:auditeur:r-x`, l'entrée `mask::rwx` et les entrées `default:`. Une mention `#effective:` signale un droit réduit par le masque.

Héritage : un fichier créé par un membre de `grp-projets-chef` dans `/srv/projets` appartient au groupe `grp-projets-dev` et porte les entrées issues de l'ACL par défaut.

Lecture seule de l'auditeur : `sudo -u auditeur touch /srv/projets/test` échoue avec `Permission denied`, `sudo -u auditeur ls /srv/projets` réussit.

Sticky bit : dans `/srv/projets/depot`, un utilisateur ne peut pas supprimer le fichier d'un collègue.

9 Dépannage

« Permission denied » malgré une ACL nommée correcte : le masque limite les droits. Vérifiez la ligne `mask::` et les annotations `#effective:` dans `getfacl`, puis rétablissez-le avec `setfacl -m m::rwx`. Vérifiez aussi le droit `x` sur chaque répertoire parent.

Les nouveaux fichiers n'ont pas le bon groupe : le SGID est absent du répertoire. Contrôlez la présence du `s` dans `ls -ld` et reposez-le avec `chmod g+s`.

Les collègues ne peuvent pas modifier les fichiers créés : le umask `022` retire l'écriture au groupe. Ajoutez une ACL par défaut avec `rwX` pour le groupe, ou réglez le umask de l'application à `002`.

« Operation not supported » avec setfacl : le système de fichiers ou le montage ne gère pas les ACL POSIX (certains montages CIFS ou FAT). Vérifiez le type avec `findmnt -T /chemin`.

Les ACL ont disparu après une restauration : l'outil de sauvegarde ne les a pas conservées. Utilisez `rsync -aAX`, `tar --acls --xattrs`, ou un outil qui les gère nativement comme BorgBackup.

10 Conseils et Bonnes Pratiques

Privilégiez les groupes. Une ACL par groupe est plus durable qu'une ACL par utilisateur. Les entrées `u:` se réservent aux exceptions temporaires et documentées.

Combinez SGID et ACL par défaut. Le SGID assure le groupe propriétaire, les ACL par défaut assurent les droits des autres groupes. Les deux ensemble rendent un répertoire collaboratif cohérent dans le temps.

Utilisez X majuscule en récursif. `rwX` donne la traversée aux répertoires sans rendre exécutables tous les fichiers de données.

Exportez les ACL avec les sauvegardes. Un export `getfacl -R` horodaté permet de restaurer les droits sans restaurer les données.

Auditez les bits SUID et SGID. Comparez périodiquement la sortie de la recherche `find -perm -4000` à une référence, et alertez sur tout nouveau binaire.

Montez les volumes de données avec nosuid et nodev. Un volume qui ne contient que des données n'a aucune raison d'honorer les bits SUID ou les fichiers de périphérique.

11 Procédures Liées

- Gestion des Utilisateurs, Groupes et de sudo *(Création des comptes et des groupes utilisés dans les ACL)*
- Gestion des Disques avec LVM et Montage par fstab *(Options de montage nosuid, nodev et noexec)*
- Installation d'un Serveur de Fichiers Samba *(Exposition des répertoires partagés aux postes Windows)*
- Déploiement d'un Serveur NFS *(Partage d'arborescences et conservation des droits)*

- Sauvegarde de Serveurs Linux avec rsync et BorgBackup *(Sauvegarde conservant les ACL)*